



*Encadrant : Pr. Abdelhamid Fadil
BADIDI Mohamed
Année universitaire 2025/2026*

Table de matière

TABLE DE MATIÈRE	2
I. COMPRÉHENSION DE LA PROBLÉMATIQUE.....	3
1. LE PROBLÈME MÉTIER	3
2. TRADUCTION EN TÂCHE ML	3
3. ÉTAT DE L'ART	3
II. ANALYSE EXPLORATOIRE DES DONNÉES (EDA)	4
1. PRÉSENTATION DU DATASET	4
2. DISTRIBUTION DE LA VARIABLE CIBLE	4
3. ANALYSE DES VARIABLES EXPLICATIVES	5
4. CORRÉLATIONS	6
III. PRÉPROCESSING DES DONNÉES	7
1. NETTOYAGE DES COLONNES	7
2. TRAITEMENT DES VALEURS ABERRANTES	7
3. VALEURS MANQUANTES ET FEATURE ENGINEERING	7
4. ENCODAGE, NORMALISATION ET PIPELINE.....	7
IV. CONSTRUCTION DES MODÈLES.....	8
V. TUNING ET CHOIX DU MODÈLE FINAL	9
1. CHOIX DES 3 MODÈLES CANDIDATS	9
2. OPTIMISATION DES HYPERPARAMÈTRES	9
3. MODÈLE FINAL ET VÉRIFICATION DE L'OVERFITTING	9
VI. DÉPLOIEMENT DU MODÈLE	10
1. SÉRIALISATION DU PIPELINE	10
2. APPLICATION STREAMLIT.....	10
3. DÉPLOIEMENT SUR STREAMLIT CLOUD.....	11
VII. CONCLUSION	12

I. Compréhension de la problématique

1. Le problème métier

L'estimation du prix d'une voiture d'occasion répond à un vrai besoin : permettre au vendeur de s'ajuster au marché avec un prix compétitif et sans grande perte, et à l'acheteur de vérifier qu'il paie un prix juste. Le prix dépend de plusieurs facteurs difficiles à évaluer manuellement, et le marché est très volatil. Un modèle de Machine Learning est donc une bonne réponse : il prend en compte toutes les caractéristiques du véhicule et s'appuie sur des données à jour pour fournir une estimation objective et reproductible. Le choix de ce problème s'est aussi basé sur mon métier, puisque je travaille dans une entreprise de data spécialisée dans le leasing automobile.

2. Traduction en tâche ML

Le problème se traduit à une tâche d'apprentissage supervisé, on dispose des prix connus. Et une régression car la variable cible : le prix est une valeur continue et non une catégorie.

Les variables explicatives sont : marque, modèle, puissance, dégâts, ...

3. État de l'art

Plusieurs familles d'algorithmes sont utilisées pour les problèmes de régression. Les **modèles linéaires** (régression linéaire, Ridge, Lasso) sont simples et interprétables... Les **modèles à base d'arbres** (Decision Tree, Random Forest)... Enfin, le **gradient boosting** (XGBoost, LightGBM)...

Pour évaluer ces modèles, on utilise principalement le MAE, le RMSE et le R^2 ...

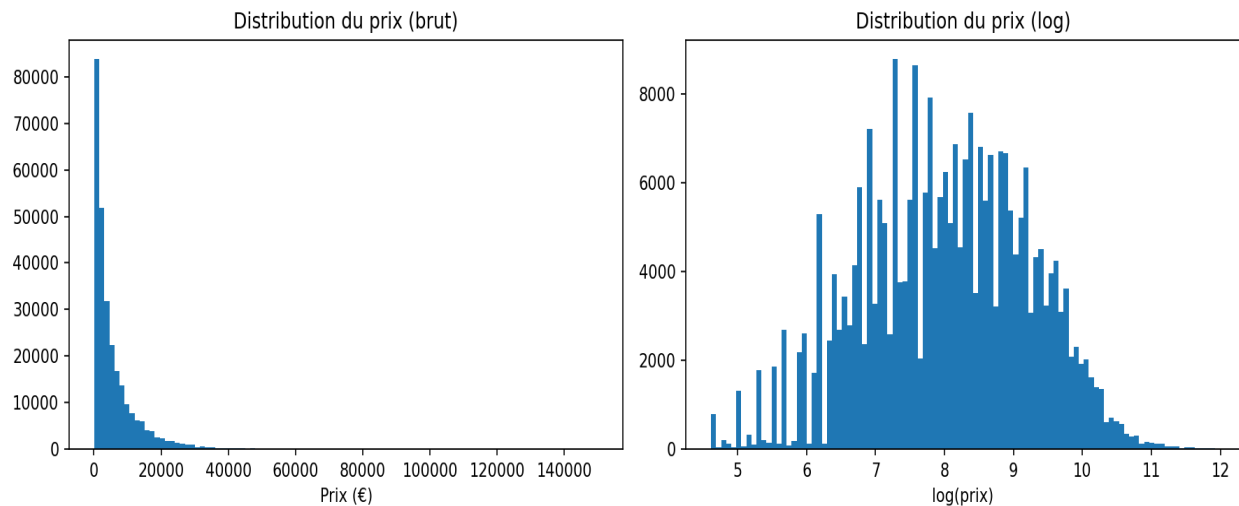
II. Analyse exploratoire des données (EDA)

1. Présentation du dataset

Les données fournis par Kaggle (lien sur PDF de l'énoncé), C'est un fichier CSV nommé autos de 351 528 annonces de vehicules d'occasions en allemagne , avec 21 colonnes dont la variable cible est : "price"

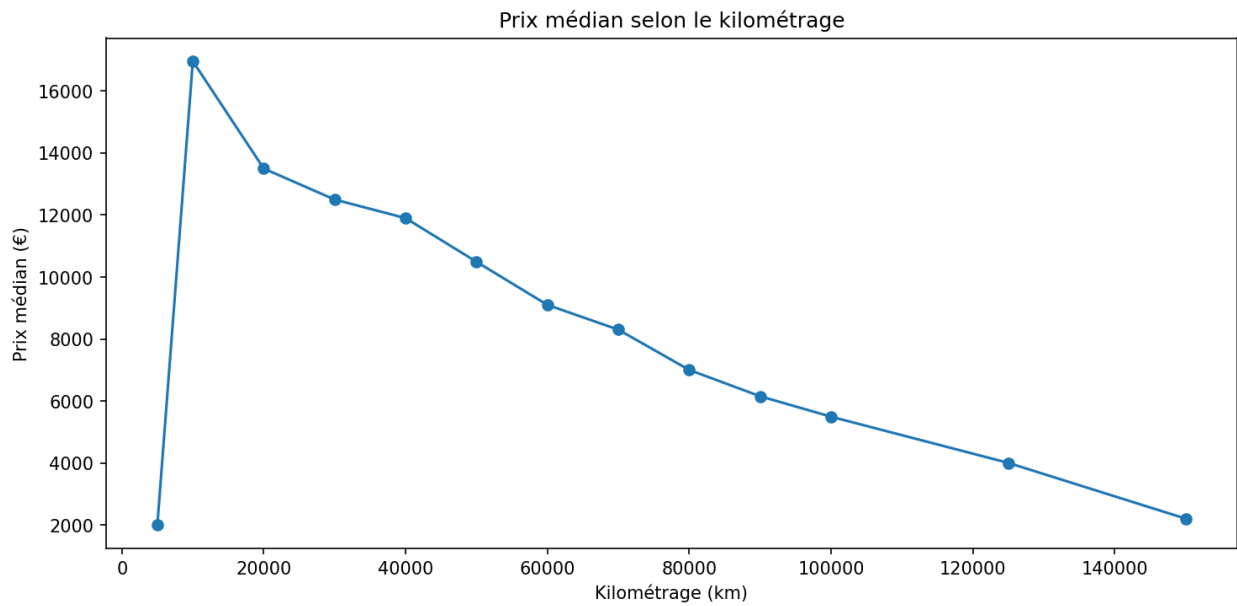
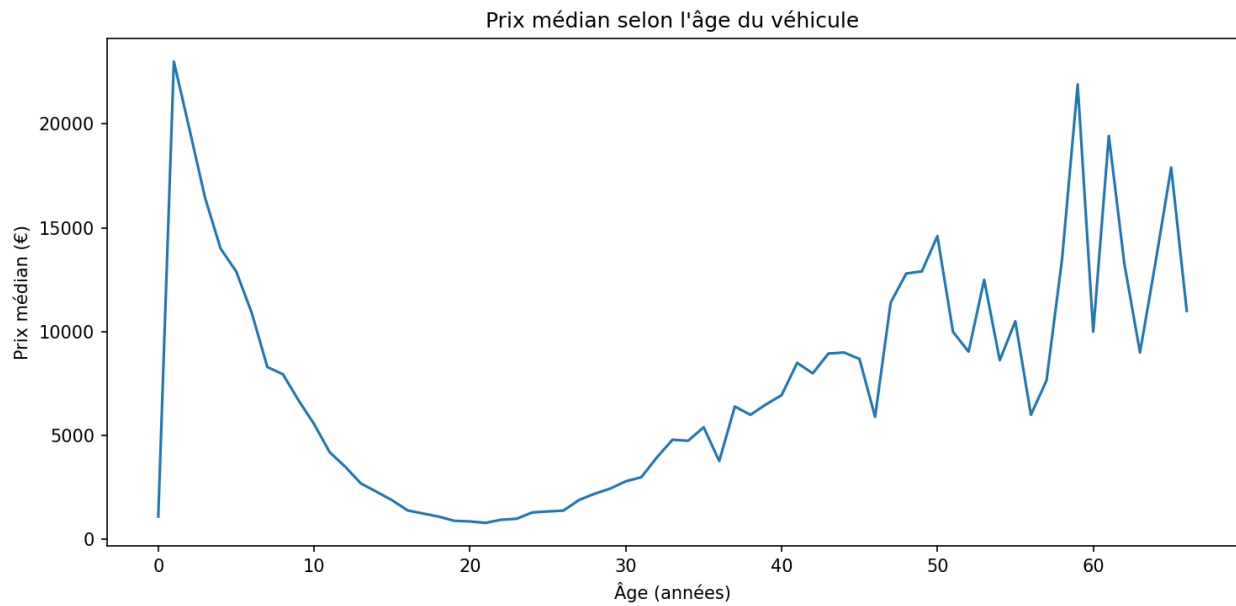
PS : le dataset diffère de la description sur l'énoncé mais peut etre utiliser pour le meme objectif : prédiction de prix

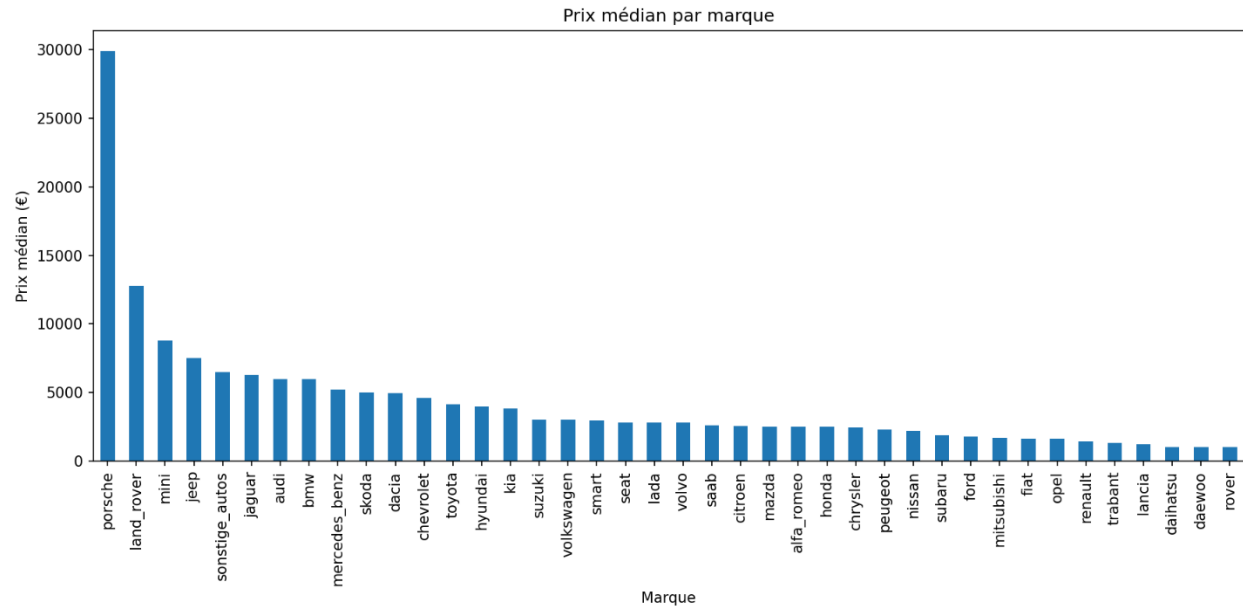
2. Distribution de la variable cible



Le prix brut est fortement asymétrique : la majorité des voitures sont bon marché et quelques-unes très chères. Après transformation log, la distribution devient quasi symétrique (en cloche). C'est pourquoi j'entraîne les modèles sur le log du prix, ce qui les aide à mieux apprendre

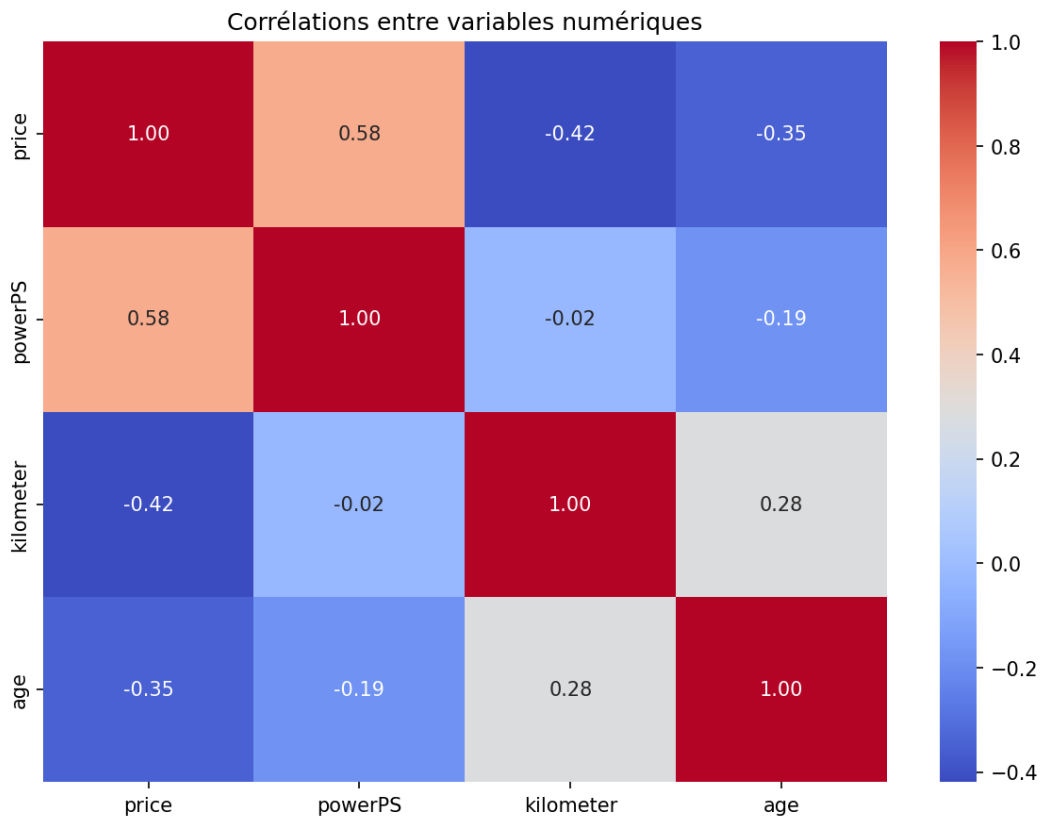
3. Analyse des variables explicatives





le prix chute fortement avec l'âge surtout les 10 premières années et aussi après les 20000 kms ; les marques premium (porsche, mercedes) ont les prix médians les plus élevés

4. Corrélations



La heatmap montre que le prix est surtout influencé par la puissance (corrélation positive +0.58 : plus la voiture est puissante, plus elle est chère). Le kilométrage (-0.42) et l'âge (-0.35) ont une corrélation négative : une voiture plus kilométrée ou plus vieille se vend moins cher. On note aussi que l'âge et le kilométrage sont liés (+0.28), ce qui est logique (les vieilles voitures ont roulé davantage).

III. Préprocessing des données

1. Nettoyage des colonnes

Avant la modélisation, on nettoie le dataset en supprimant deux types de colonnes. D'abord les colonnes constantes ou quasi constantes : `nrOfPictures` (toujours égale à 0), `seller` et `offerType` (une seule valeur domine à plus de 99 %). Ces colonnes ne permettent pas de distinguer les voitures entre elles. Ensuite, les colonnes inutiles pour prédire le prix : les dates de scraping (`dateCrawled`, `dateCreated`, `lastSeen`), l'identifiant `index`, le `postalCode` et `monthOfRegistration`.

Ce sont des décisions de métiers.

2. Traitement des valeurs aberrantes

Après analyse on peut repérer des valeurs aberrantes qu'on doit traiter avant la modélisation, comme exemples : des prix de 0 à 2 milliards de euros des puissances de 0 à 2000ch, des années d'immatriculation de 1000 à 9999.

Ce qu'on fait c'est compter combien de lignes montrent chaque anomalie, filtrer pour garder que des valeurs réalistes. Et pour les valeurs nombreuses les transformer pour les imputer.

On garde les prix de 100 à 150 000 euros et des dates de 1950 à 2016 (les dates de scraping les plus récentes), et des puissances moins que 1000ps

Ce sont des décisions de métiers.

3. Valeurs manquantes et feature engineering

Les puissances à 0, impossibles en réalité, correspondent en fait à des valeurs non renseignées. Comme elles sont nombreuses, on les transforme en valeurs manquantes au lieu de supprimer les lignes, afin de les imputer ensuite. L'imputation se fait par la médiane pour les variables numériques et par une catégorie « unknown » pour les variables catégorielles. Enfin, on crée la variable âge (2016 – année d'immatriculation), plus parlante que l'année brute et plus directement liée au prix.

4. Encodage, normalisation et pipeline

Les variables catégorielles sont encodées en One-Hot : chaque catégorie devient une colonne 0/1, car un modèle ne traite que des nombres. Les variables numériques sont normalisées avec `StandardScaler` (moyenne 0, écart-type 1) pour les mettre à la même échelle. La variable cible, fortement asymétrique, est transformée par le logarithme (`log1p`) puis reconvertie en euros (`expm1`) pour l'évaluation. Tous ces traitements sont assemblés dans un `ColumnTransformer`,

qui applique automatiquement le bon traitement à chaque type de colonne et garantit un préprocessing reproductible.

IV. Construction des modèles

On commence par une régression linéaire comme modèle de référence (baseline) car c'est le modèle le plus simple ; elle sert de point de comparaison.

On obtient : MAE = 2 402 €, RMSE = 8 652 €, $R^2 = -0.161$.

Ce R^2 négatif est anormal, je vérifie donc pourquoi : en regardant les prédictions extrêmes, je constate que la régression linéaire extrapole trop loin (jusqu'à 800 000 €), ce qui fausse le score.

Ensuite, on teste 11 algorithmes de familles différentes :

- Arbres : DecisionTree
- Ensembles (bagging) : RandomForest, ExtraTrees
- Boosting : GradientBoosting, XGBoost, LightGBM
- Linéaires : LinearRegression, Ridge, Lasso, ElasticNet
- Voisinage : KNN

On obtient les résultats suivants :

	Modèle	R2_log	RMSE_log	MAE_eur	R2_eur	Temps_s
0	XGBoost	0.850029	0.469061	1382.236328	0.827617	3.2
1	RandomForest	0.849381	0.470072	1225.280274	0.848474	1102.6
2	LightGBM	0.842329	0.480951	1451.637019	0.802541	3.1
3	ExtraTrees	0.841237	0.482614	1226.889738	0.844031	1275.3
4	KNN	0.830183	0.499132	1410.340969	0.820534	517.1
5	GradientBoosting	0.813580	0.522963	1640.240215	0.753011	33.7
6	DecisionTree	0.792590	0.551620	1401.678651	0.779815	107.0
7	LinearRegression	0.705108	0.657743	2332.433209	0.336716	4.8
8	Ridge	0.705093	0.657760	2330.989148	0.339562	3.9
9	ElasticNet	0.096202	1.151492	4375.941694	-0.077908	2.1
10	Lasso	-0.000011	1.211232	4534.422904	-0.130446	1.6

V. Tuning et choix du modèle final

1. Choix des 3 modèles candidats

Après tester les 11 modèles , les mieux classés (R^2_{log}): XGBoost, RandomForest, LightGBM , (on a aussi une diversité entre le bagging et le boosting).

2. Optimisation des hyperparamètres

on fait un tuning on changeant les hyperparametres deja fixées en random et de cross-validation pour enfin choisir un modèle final en se basant sur la performance et l'overfitting.

Et on trouve les résultats :

	Modèle	R2_log_train	R2_log_test	MAE_eur	RMSE_eur	R2_eur_test
0	XGBoost (tuné)	0.890371	0.861730	1252.654907	3102.101546	0.850805
1	LightGBM (tuné)	0.882963	0.861304	1248.948118	3070.154088	0.853862
2	RandomForest (tuné)	0.924179	0.853277	1268.455997	3316.249489	0.829495

3. Modèle final et vérification de l'overfitting

On choisit LightGBM pour ces meilleures métriques de test et plus faible overfitting (écart train/test de 0.022, contre 0.071 pour RandomForest qui surapprend).

VI. Déploiement du modèle

1. Sérialisation du pipeline

On sauvegarde le PIPELINE COMPLET (préprocessing + modèle) pour que le préprocessing se dait automatiquement et aussi pour que l'application Streamlit recoit que Rawdata et le pipeline fait tout l'imputation + normalisation + encodage + prédiction.

2. Application Streamlit

L'application Streamlit permet à l'utilisateur de saisir les caractéristiques d'une voiture via des menus déroulants (en français) et des curseurs. Le menu « modèle » s'adapte automatiquement à la marque choisie.

Estimation du prix d'une voiture d'occasion

Renseignez les caractéristiques du véhicule pour obtenir une estimation de prix.

Caractéristiques du véhicule

Marque	Carburant
alfa_romeo	andere
Modèle	Dégât non réparé
145	ja
Type de véhicule	Puissance (ch)
andere	116
Boîte de vitesses	Kilométrage (km)
automatik	100000
	Âge du véhicule (années)
	13

Un tableau récapitule les données saisies, puis le bouton « Estimer le prix » affiche la prédiction.

Données saisies

	vehicleType	gearbox	powerPS	model	kilometer	fuelType	brand	notRepairedDamage
0	andere	automatik	116	145	100000	andere	alfa_romeo	ja

Estimer le prix

L'application fournit également un intervalle de confiance ($\approx 80\%$) grâce à deux modèles de quantile (10 % et 90 %) : comme une régression ne donne pas de probabilité, cet intervalle traduit l'incertitude de l'estimation.

Résultat

Prix estimé

846 €

Intervalle de confiance ($\sim 80\%$) : 316 € — 2,546 €

Estimation basée sur un modèle LightGBM entraîné sur ~340 000 annonces.

3. Déploiement sur Streamlit Cloud

Le projet a été déposé sur un dépôt GitHub public contenant l'ensemble des livrables. Le dépôt est structuré ainsi :

- app.py : le code de l'application Streamlit
- notebook.ipynb : le notebook complet (EDA, préprocessing, modélisation, évaluation)
- model_pipeline.joblib : le pipeline sérialisé (préprocessing + modèle + modèles de quantile)
- app_options.joblib : les options des menus de l'application
- requirements.txt : les dépendances nécessaires au déploiement
- README.md : la description du projet, le lien vers l'application et les captures d'écran

L'application a ensuite été déployée sur Streamlit Cloud, ce qui la rend accessible publiquement via une URL.

Dépôt GitHub : https://github.com/badidiM/ML_project_carprice_prediction

Application déployée : <https://mlprojectcarpriceprediction-59nbw8rhsjxxcdslrappcfa.streamlit.app/>

VII. Conclusion

Ce projet a permis de mettre en œuvre un pipeline de Machine Learning complet, de la compréhension du problème métier jusqu'au déploiement d'une application. Après avoir testé 11 algorithmes et optimisé les meilleurs, le modèle final retenu est LightGBM, avec un R^2 d'environ 0.85 et une erreur moyenne d'environ 1 250 €.

Ce travail présente toutefois des limites. Le modèle a été entraîné sur des données du marché allemand de 2016, il n'est donc pas directement transposable au marché actuel. De plus, l'application autorise des combinaisons de saisie irréalistes, car elle ne vérifie pas la cohérence technique entre les caractéristiques.

Plusieurs pistes d'amélioration sont envisageables : tester le target encoding pour la variable model, utiliser des données plus récentes et adaptées au marché ciblé, et ajouter des contrôles de cohérence dans l'application.